W5.2 D3 Pie and stack charts

Lecture

Agenda

- 5.1 Lines
- 5.2 Pie and stack

Class

Agenda

- D3 Lines and Curves
- D3 Pie and Stack

Layout function

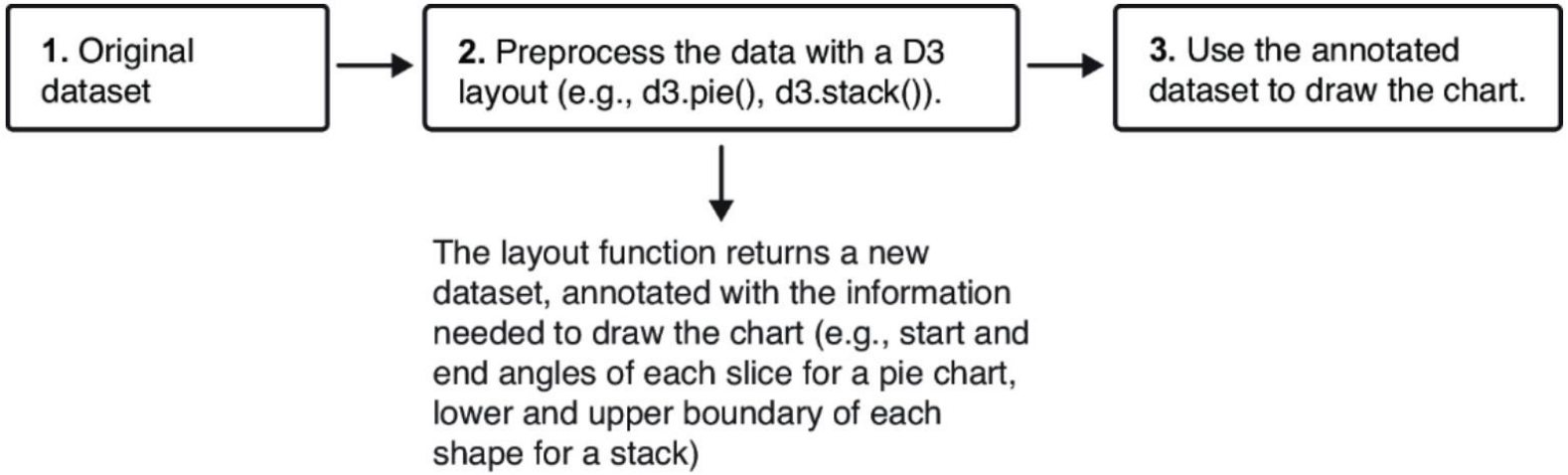


Figure 5.1 The layout function is a data preprocessing step used to calculate the information required to draw a specific chart.

SWINBURNE UNIVERSITY OF TECHNOLOGY

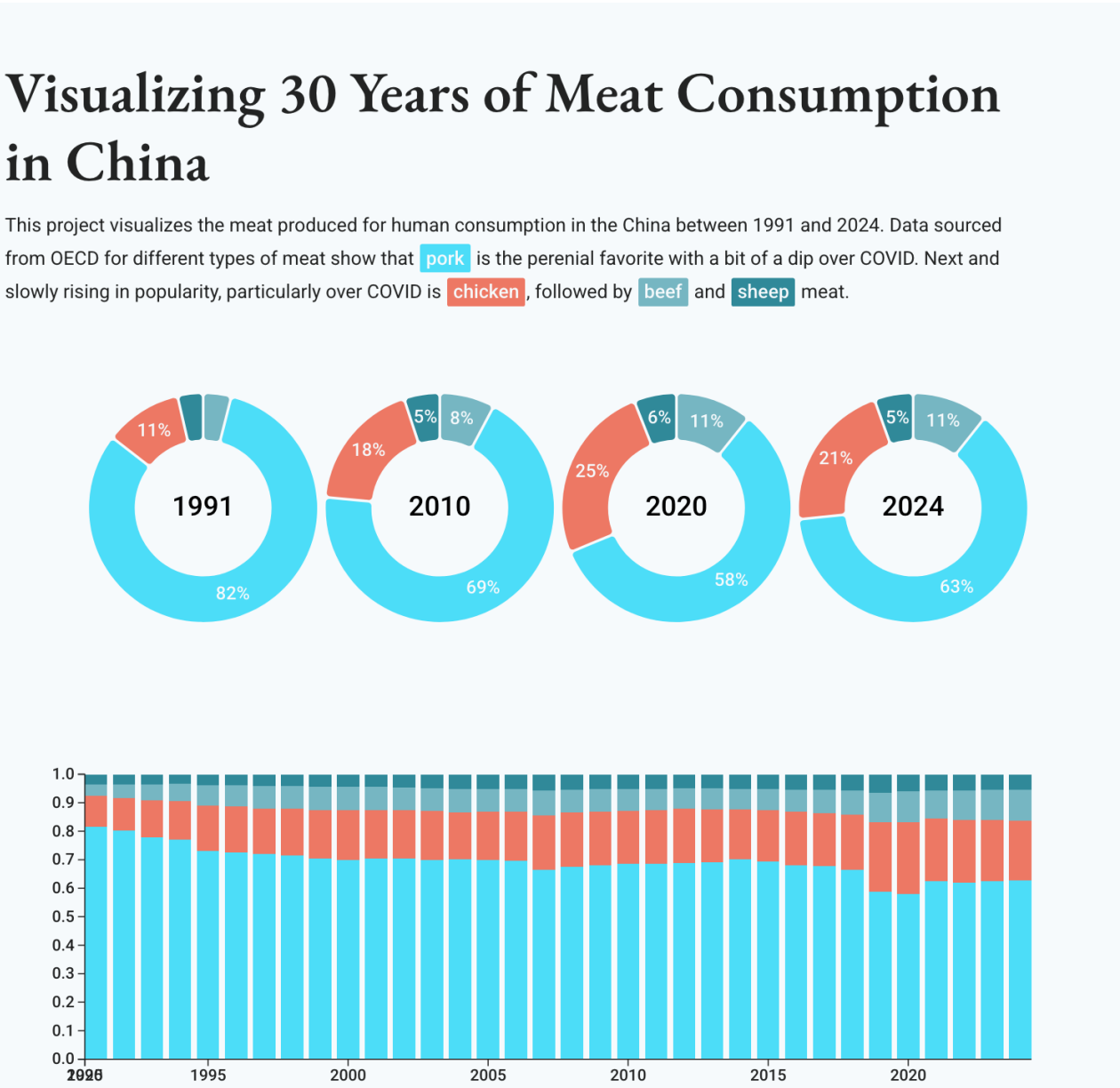
Charts to build

Donut/Pie Chart

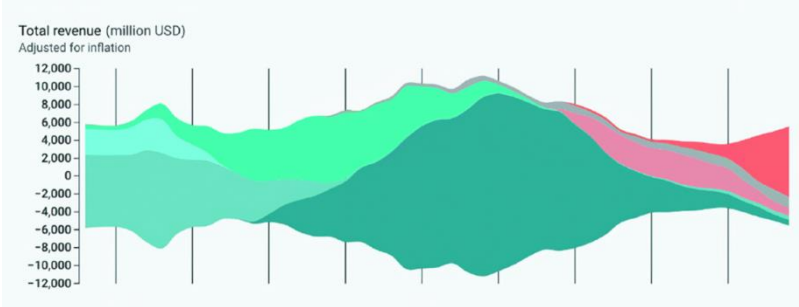
- For proportions of categories
- No more than 4-5 categories

Stacked Bar Chart

- Relative differences



Dufour & Meeks (2024)
Also build streamgraph



Dufour & Meeks (2024)

Laying out charts on page

Shared constants for svg containers

```
js > JS shared-constant.js > ...
1   const margin = {top: 50, right: 0, bottom: 50, left: 70};
2   const width = 900;
3   const height = 350;
4   const innerWidth = width - margin.left - margin.right;
5   const innerHeight = height - margin.top - margin.bottom;
6
7   const formatsInfo = [
8     {id: "beef", label: "Beef", color: "#76B6C2"},
9     {id: "pig", label: "Pork", color: "#4CDDF7"},
10    {id: "poultry", label: "Poultry", color: "#ED7864"},
11    {id: "sheep", label: "Sheep", color: "#2F8999"},
12  ];
```



Make sure you call this file in index.html

```
<!-- Load your scripts here -->
<script src="https://d3js.org/d3.v7.min.js"></script>
<script src="js/shared-constants.js"></script>
<script src="js/scales.js"></script>
<script src="js/donut-charts.js"></script>
<script src="js/stacked-bars.js"></script>
<script src="js/legend.js"></script>
<script src="js/load-data.js"></script>
```

```
1   year,beef,pig,poultry,sheep
2   1991,0.7055,14.5729,1.9485,0.6401
3   1992,0.9413,15.5892,2.1968,0.6709
4   1993,1.1809,16.6971,2.7738,0.7295
5   1994,1.4318,18.084,3.1468,0.7795
6   1995,1.8363,18.3251,3.9881,0.9115
7   1996,1.8281,17.9695,3.9731,0.9387
8   1997,2.2592,20.3398,4.5264,1.0952
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Load data – put in separate file

```
d3.csv("../data/chinameat.csv", d => ({
  year: +d.year,
  beef: +d.beef,
  pig: +d.pig,
  poultry: +d.poultry,
  sheep: +d.sheep
})).then(data => {
console.log("Parsed data:", data); //  Check if 'year' is being parsed correctly
  defineScales(data);
  drawDonutCharts(data);
  drawStackedBars(data);
  addLegend();
});
```

Index.html

```
<div id="donut"></div>
<div id="bars"></div>
<div class="legend-container"></div>
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Add svg container and group for chart

```
const svg = d3.select("#donut")
  .append("svg")
  .attr("viewBox", `0 0 ${width} ${height}`);

const donutContainers = svg
  .append("g")
  .attr("transform", `translate(${margin.left}, ${margin.top})`);
```

Index.html

```
<div id="donut"></div>
<div id="bars"></div>
<div class="legend-container"></div>
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

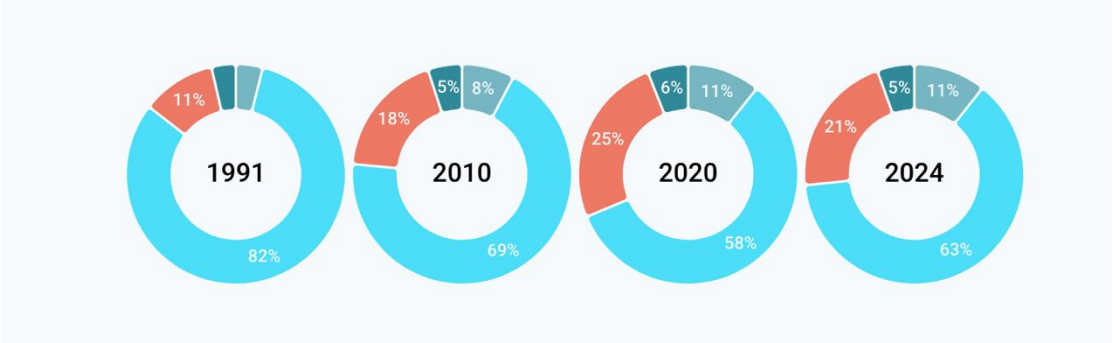
Using different containers for each set of charts

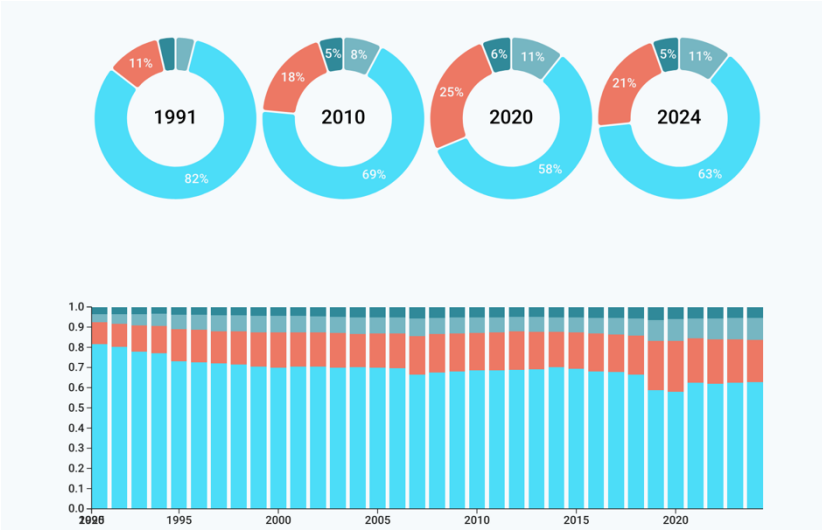
We want chart for selected years

- Earliest year, most recent year
- COVID
- Year between COVID and start date

X-axis position to be defined by date

```
<div id="donut"></div>
<div id="bars"></div>
```





Dufour & Meeks (2024)

Positioning

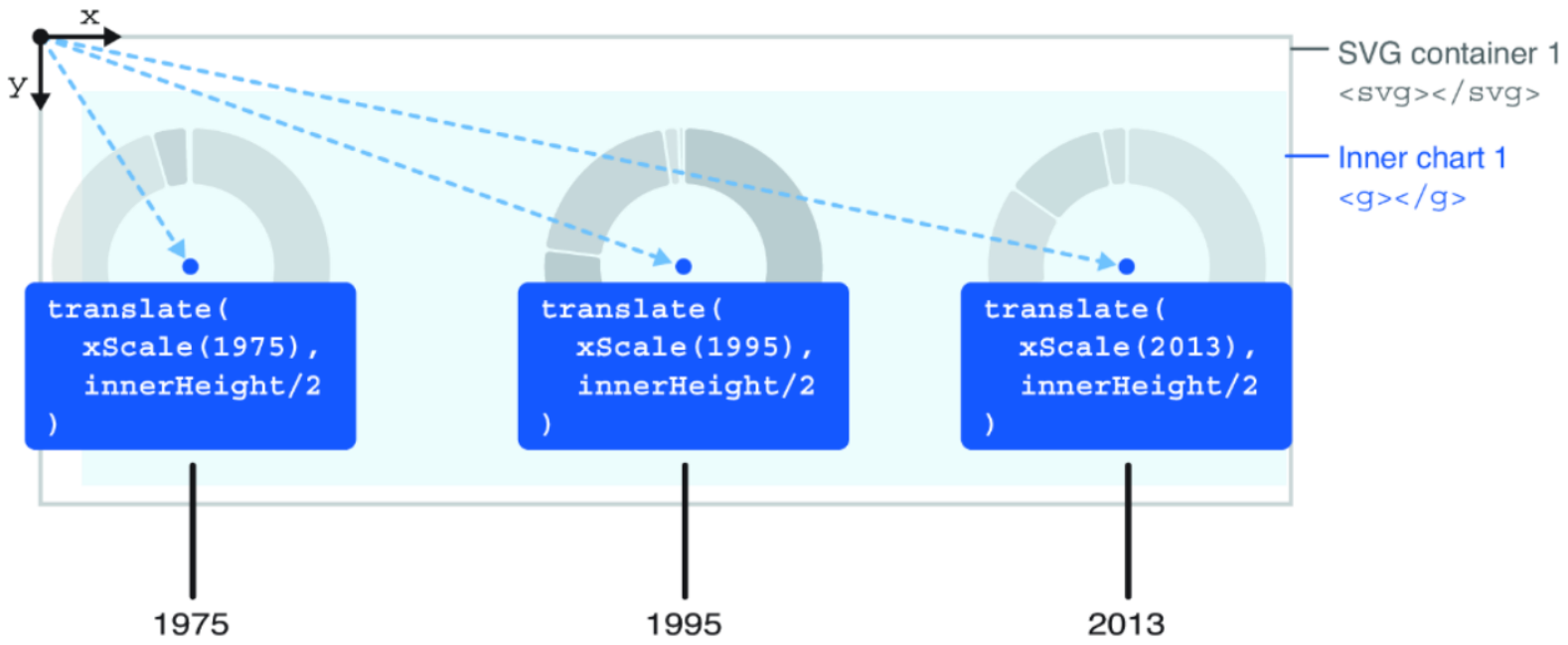


Figure 5.4 Each set of arcs composing a donut chart is contained inside an SVG group. These groups are translated horizontally based on the year they represent. This position is calculated with a D3 scale.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Declare xScale in separate scales.js file

X-axis position will be based on date, treat as category

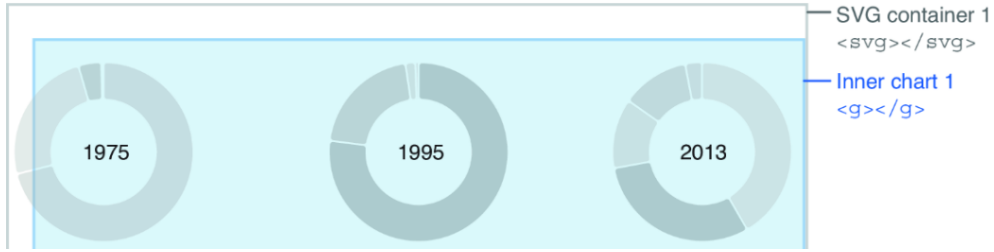
Use band scale

```
const xScale = d3.scaleBand();

const defineScales = (data) => {

  xScale
    .domain(data.map(d => d.year))
    .range([0, innerWidth])

};
```





Setting defineScales lets data load before calling scales

```
d3.csv("../data/chinameat.csv", row => ({
  year: +row.year,
  beef: +row.beef,
  pig: +row.pig,
  poultry: +row.poultry,
  sheep: +row.sheep
})).then(data => {
  console.log("Parsed data:", data);
  defineScales(data);
  drawDonutCharts(data);
  drawStackedBars(data);
  drawStreamGraph(data);
  addLegend();
});
```

DrawDonutCharts

Add a group for each year and translate it relative to year

```
const years = [1991, 2010, 2020, 2024];

  years.forEach(year => {

    const donutContainer = donutContainers
      .append("g")
      .attr("transform", `translate(${xScale(year)}, ${innerHeight/2})`);
```

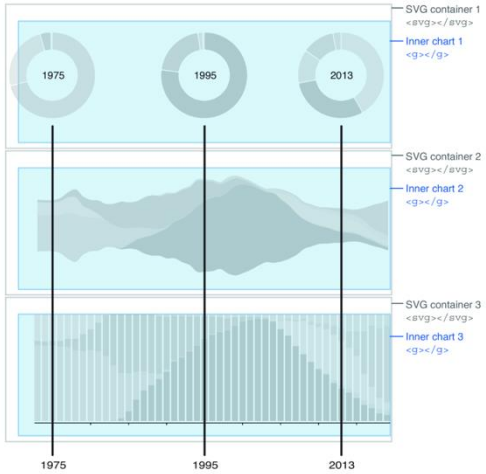


Figure 5.3 To create this chapter's project, we'll use three SVG containers: one for the donut charts, one for the streamgraph, and one for the stacked bar chart. This strategy will allow us to have a consistent area reserved for the inner chart and to align each chart on top of the other properly.

Or translate relative to number of charts being drawn (GitHub Copilot)

```
const chartSpacing = (width - margin.left - margin.right) / years.length;
const chartOffset = chartSpacing / 2;

const donutContainer = donutContainers
  .append("g")
  .attr("transform", `translate(${chartOffset + index * chartSpacing}, ${innerHeight / 2})`);
```

Space each chart can take up

SWINBURNE UNIVERSITY OF TECHNOLOGY

Constants for Pie generator

Needs an array of numbers to calculate radians

We also need to connect each number to its category

We can get category name from column

We can get column names from **data.columns**

We don't need year to draw chart, but years tells us which years to draw charts for in **forEach** loop):




```
const years = [1991, 2010, 2020, 2024];
```

Will tell generator what years to draw chart for

```
const formats = data.columns.filter(format => format !== "year");
```

Will tell generator **not** to use year to calculate angles

```
formats = ["beef", "pig", "poultry", "sheep"];
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Format data for pie generator

```
▶ 33: Object { year: 2024, beef: 4.9904, pig: 29.3815, … }
▶ columns: Array(5) [ "year", "beef", "pig", … ]
  length: 34
▶ <prototype>: Array []
```

```
formattedData ▼ Array(4) [ {…}, {…}, {…}, {…} ]
  ▶ 0: Object { format: "beef", production: 0.7055 }
  ▶ 1: Object { format: "pig", production: 14.5729 }
  ▶ 2: Object { format: "poultry", production: 1.9485 }
  ▶ 3: Object { format: "sheep", production: 0.6401 }
    length: 4
  ▶ <prototype>: Array []
```

```
formattedData ▼ Array(4) [ {…}, {…}, {…}, {…} ]
  ▶ 0: Object { format: "beef", production: 2.9889 }
  ▶ 1: Object { format: "pig", production: 26.4847 }
  ▶ 2: Object { format: "poultry", production: 7.1149 }
  ▶ 3: Object { format: "sheep", production: 1.9391 }
    length: 4
```

```
const years = [1991, 2010, 2020, 2024];
const formats = data.columns.filter(format => format !== "year");

years.forEach((year, index) => {

  const donutContainer = donutContainers
   .append("g")
   .attr("transform", `translate(${chartOffset + index * chartSpacing}, ${innerHeight / 2})`);


  const yearData = data.find(d => d.year === year);

  const formattedData = [];

  formats.forEach(format => {
    formattedData.push({
      format: format,
      production: yearData[format]
      });
  });

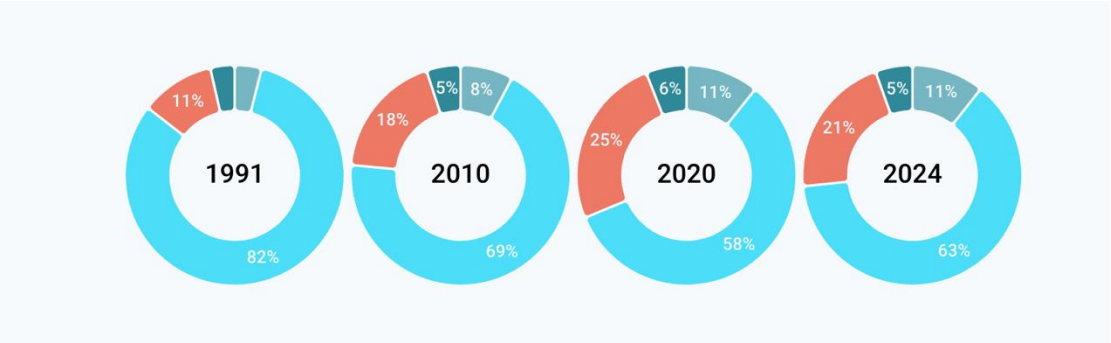
  console.log("formattedData", formattedData);
```

Creates an array for each specified year

```
[
  { format: "beef", production: 0.7055 },
  { format: "pig", production: 14.5729 },
  { format: "poultry", production: 1.9485 },
  { format: "sheep", production: 0.6401 }
];
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Create a pie generator



Create pie generator

For each year will get meat production data

Automatically sorted, but for ease of comparison we want each meat type in same spot (so null sort)

```
years.forEach((year, index) => {
...
    const pieGenerator = d3.pie()
      .value(d => d.production)
      .sort(null);
```

Function for producing data in format required to create arcs

To get data to generate chart (eg start and end angles)

```
    const annotatedData = pieGenerator(formattedData);

});
```

```
[
  {
    data: { format: "beef", production: 100 },
    index: 0,
    value: 100,
    startAngle: 0,
    endAngle: 0.6283185307179586, // (100 / 500) * 2π
    padAngle: 0
  },
  {
    data: { format: "pig", production: 200 },
    index: 1,
    value: 200,
    startAngle: 0.6283185307179586,
    endAngle: 2.5132741228718345, // (300 / 500) * 2π
    padAngle: 0
  },
  {
    data: { format: "poultry", production: 150 },
    index: 2,
    value: 150,
    startAngle: 2.5132741228718345,
    endAngle: 4.39822971502571, // (450 / 500) * 2π
    padAngle: 0
  },
  {
    data: { format: "sheep", production: 50 },
    index: 3,
    value: 50,
    startAngle: 4.39822971502571,
    endAngle: 6.283185307179586, // (500 / 500) * 2π
    padAngle: 0
  }
]
```

Overview of getting data required to draw arcs in pie chart

1. Format the data.
2. Initialize the pie layout function.
3. Call the pie layout and pass the formatted data as an argument. Later, we'll use the annotated dataset returned by the pie layout to draw the arcs.

1. Format the data to an array of values or an array of objects containing these values.

```
formattedData = [
  { item: "item_A", value: 60 },
  { item: "item_B", value: 30 },
  { item: "item_C", value: 10 }
];
```

2. Initialize a pie layout generator with the method `d3.pie()`. If the data is formatted as an array of objects, use the `value()` accessor function to specify the key in which the values are contained.

```
const pieGenerator = d3.pie()
  .value(d => d.value);
```

3. Call the pie generator function and pass the formatted data as an argument.

Store the array returned by the pie generator into a constant. **This is the annotated dataset you'll use later to draw the arcs of the pie or donut chart.**

```
const annotatedData
  = pieGenerator(formattedData)
```

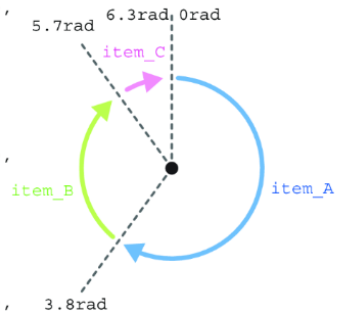
Annotated Dataset

```
[
  {
    data: { item: "item_A", value: 60 },
    value: 60,
    index: 0,
    startAngle: 0,
    endAngle: 3.8
  },
  {
    data: { item: "item_B", value: 30 },
    value: 30,
    index: 0,
    startAngle: 3.8,
    endAngle: 5.7
  },
  {
    data: { item: "item_C", value: 10 },
    value: 10,
    index: 0,
    startAngle: 5.7,
    endAngle: 6.3,
  }
];
```

A

B

C



Dufour & Meeks (2024)

SWINBURNE UNIVERSITY OF TECHNOLOGY

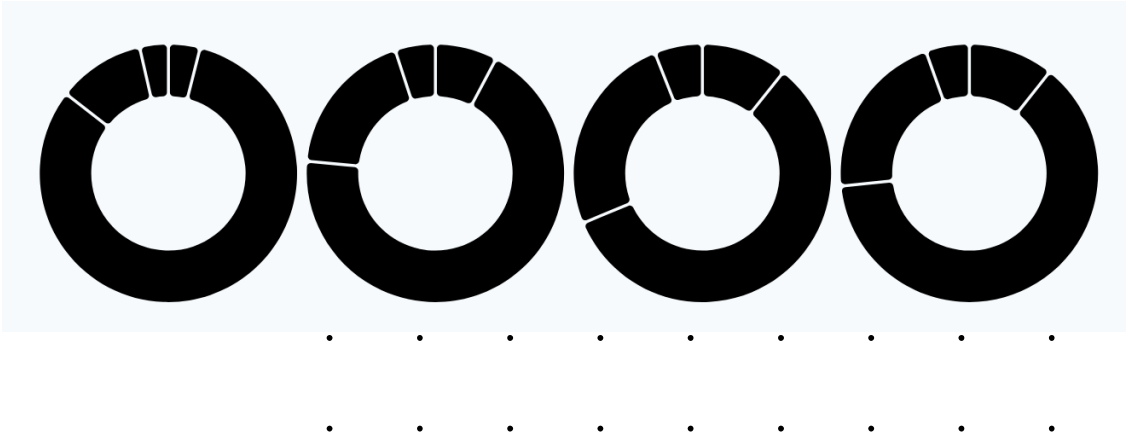
Drawing arcs

```
const arcGenerator = d3.arc()
  .startAngle(d => d.startAngle)
  .endAngle(d => d.endAngle)
  .innerRadius(60)
  .outerRadius(100)
  .padAngle(0.02)
  .cornerRadius(3);

const arcs = donutContainer
  .selectAll(`.arc-${year}`)
  .data(annotatedData)
  .join("path")
    .attr("class", `arc-${year}`)
    .attr("d", arcGenerator);
```

Function for producing data in format required to **drawing** arcs (paths)

```
[
    {
        data: { format: "beef", production: 100 },
        index: 0,
        value: 100,
        startAngle: 0,
        endAngle: 0.6283185307179586, // (100 / 500) * 2π
        padAngle: 0
    },
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Define Colour scale

scales.js

```
const xScale = d3.scaleBand();
const colorScale = d3.scaleOrdinal();

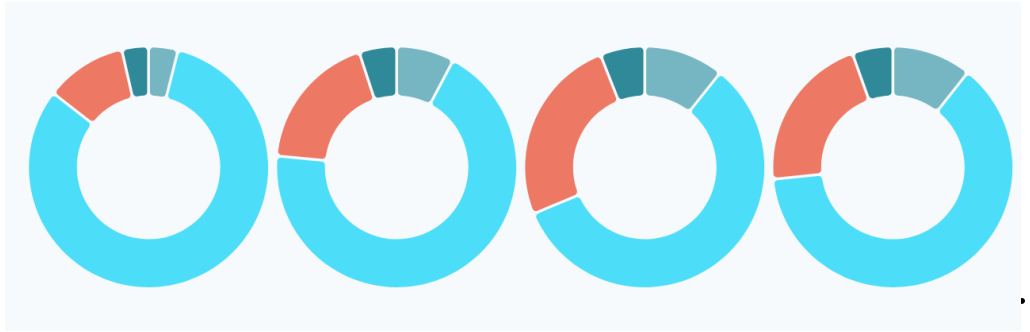
const defineScales = (data) => {
...
        colorScale
        .domain(formatsInfo.map(f => f.id))
        .range(formatsInfo.map(f => f.color));
};
```

shared-constants.js

```
const formatsInfo = [
  {id: "beef", label: "Beef", color: "#76B6C2"},
  {id: "pig", label: "Pork", color: "#4CDDF7"},
  {id: "poultry", label: "Poultry", color: "#ED7864"},
  {id: "sheep", label: "Sheep", color: "#2F8999"},
```

donut_charts.js

```
const arcs = donutContainer
  .selectAll(`.arc-${year}`)
  .data(annotatedData)
  .join("path")
    .attr("class", `arc-${year}`)
    .attr("d", arcGenerator)
    .attr("fill", d => colorScale(d.data.format));
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Adding labels

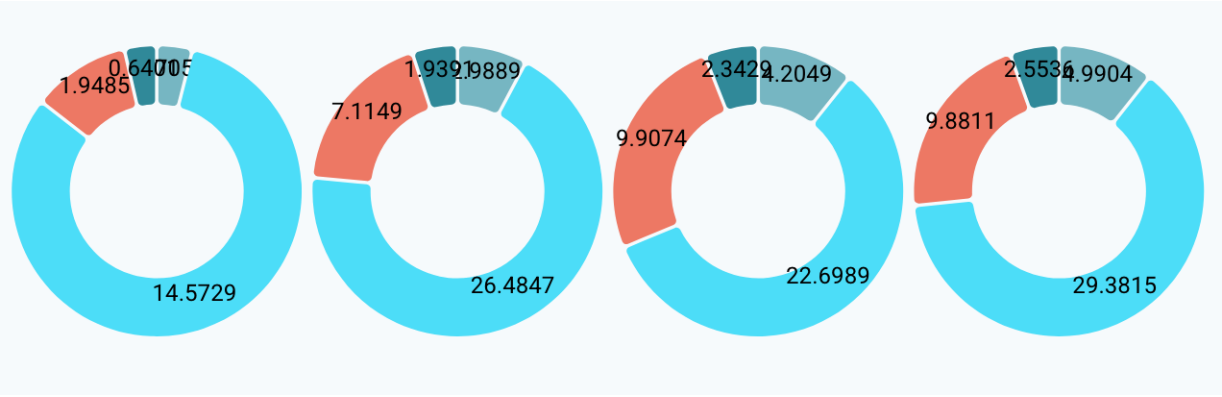
Change join to a group

Add arc

Add text

```
const arcs = donutContainer
  .selectAll(`.arc-${year}`)
  .data(annotatedData)
  .join("g")
    .attr("class", `arc-${year}`)
  arcs
    .append("path")
      .attr("d", arcGenerator)
      .attr("fill", d => colorScale(d.data.format));
  arcs
   .append("text")
    .attr("transform", d => `translate(${arcGenerator.centroid(d)})`)
    .attr("text-anchor", "middle")
    .attr("alignment-baseline", "middle")
    .text(d => d.data.production);
```

GitHub Copilot auto complete



SWINBURNE UNIVERSITY OF TECHNOLOGY

Add nice labels

Calculate percentage of each arc of total circle

Value stored using bracket notation (JavaScript)

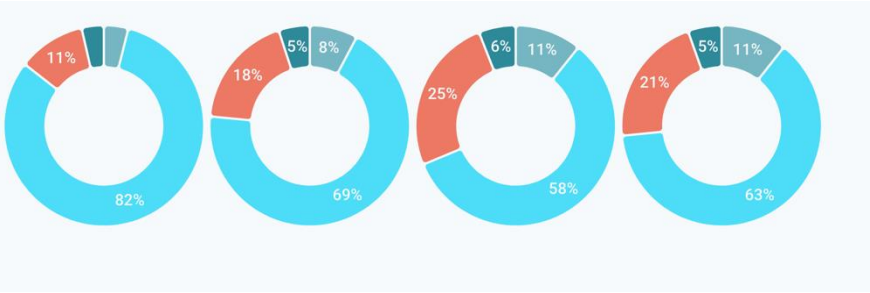
Format as percentage string (e.g., 25%) which becomes label

Calculate centroid 'manually' and use for x and y

Nice to have:

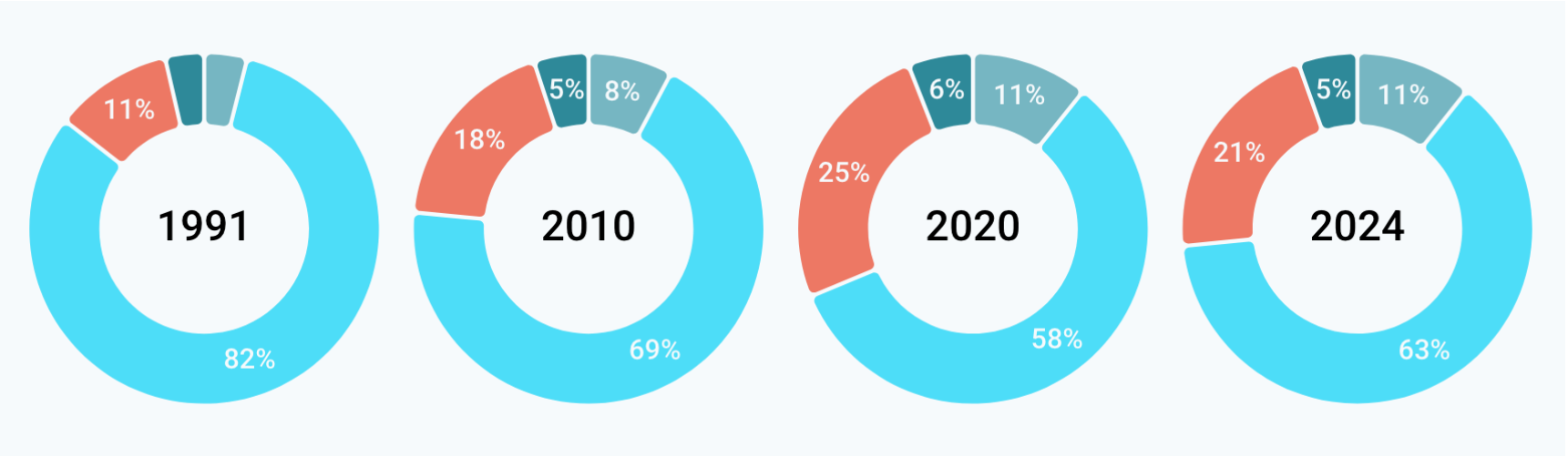
- Set opacity to 0 when percent is smaller than 5%

```
arcs
 .append("text")
  .text(d => {
    d["percentage"] = (d.endAngle - d.startAngle)/ (2 * Math.PI);
    return d3.format(".0%")(d.percentage);
  })
  .attr("x", d => {
    d["centroid"] = arcGenerator
      .startAngle(d.startAngle)
      .endAngle(d.endAngle)
      .centroid();
    return d.centroid[0];
  })
  .attr("y", d => d.centroid[1])
  .attr("text-anchor", "middle")
  .attr("dominant-baseline", "middle")
  .attr("fill", "#f6fafc")
  .attr("fill-opacity", d => d.percentage< 0.05 ? 0 : 1)
  .style("font-size", "16px")
  .style("font-weight", 500);
```



Add year labels to center

```
donutContainer
  .append("text")
    .text(year)
    .attr("text-anchor", "middle")
    .attr("dominant-baseline", "middle")
    .style("font-size", "24px")
    .style("font-weight", 500);
```



Dufour & Meeks (2024)

Creating pie or donut chart

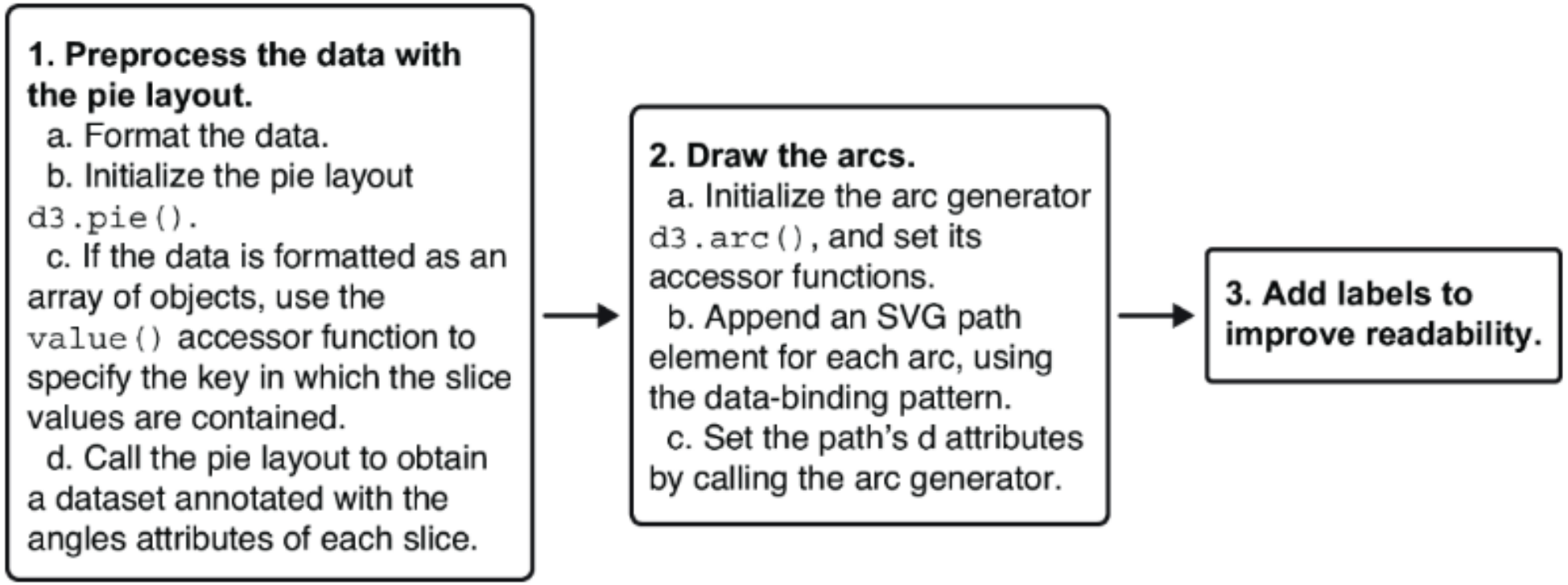


Figure 5.10 The main steps involved in the creation of a pie or donut chart

SWINBURNE UNIVERSITY OF TECHNOLOGY

W5.2 D3 Pie and stack charts

Lecture

Agenda

- 5.1 Lines
- 5.2 Pie and stack

Class

Agenda

- D3 Lines and Curves
- D3 Pie and Stack

Stacked bar chart

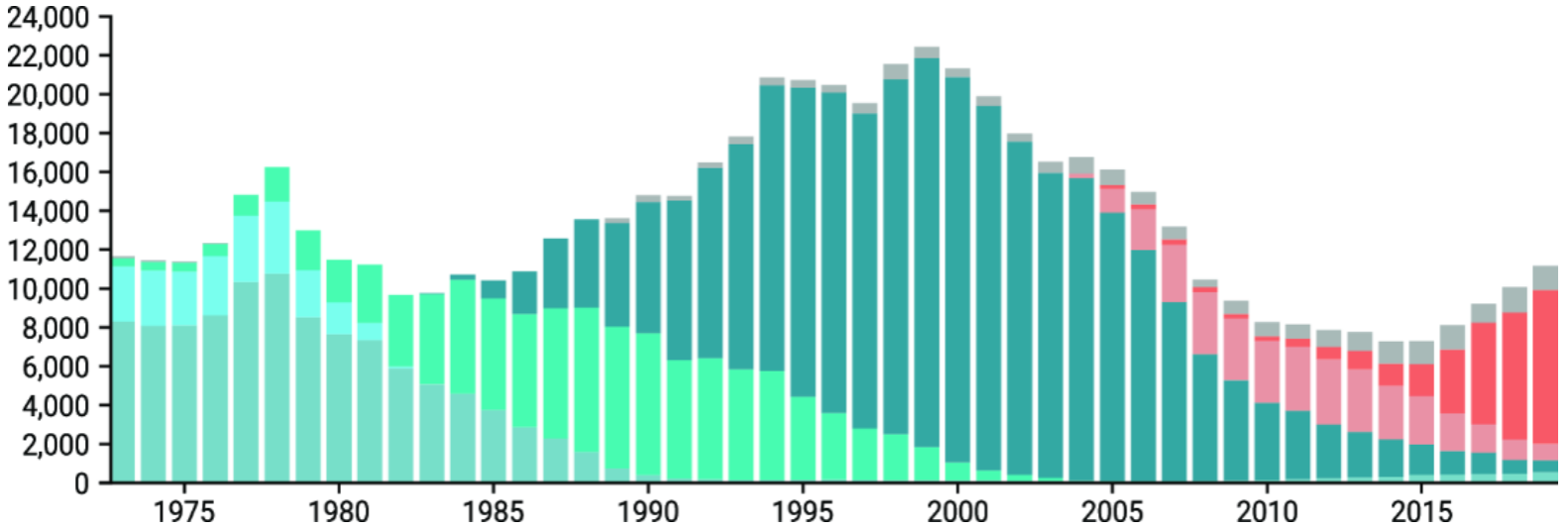
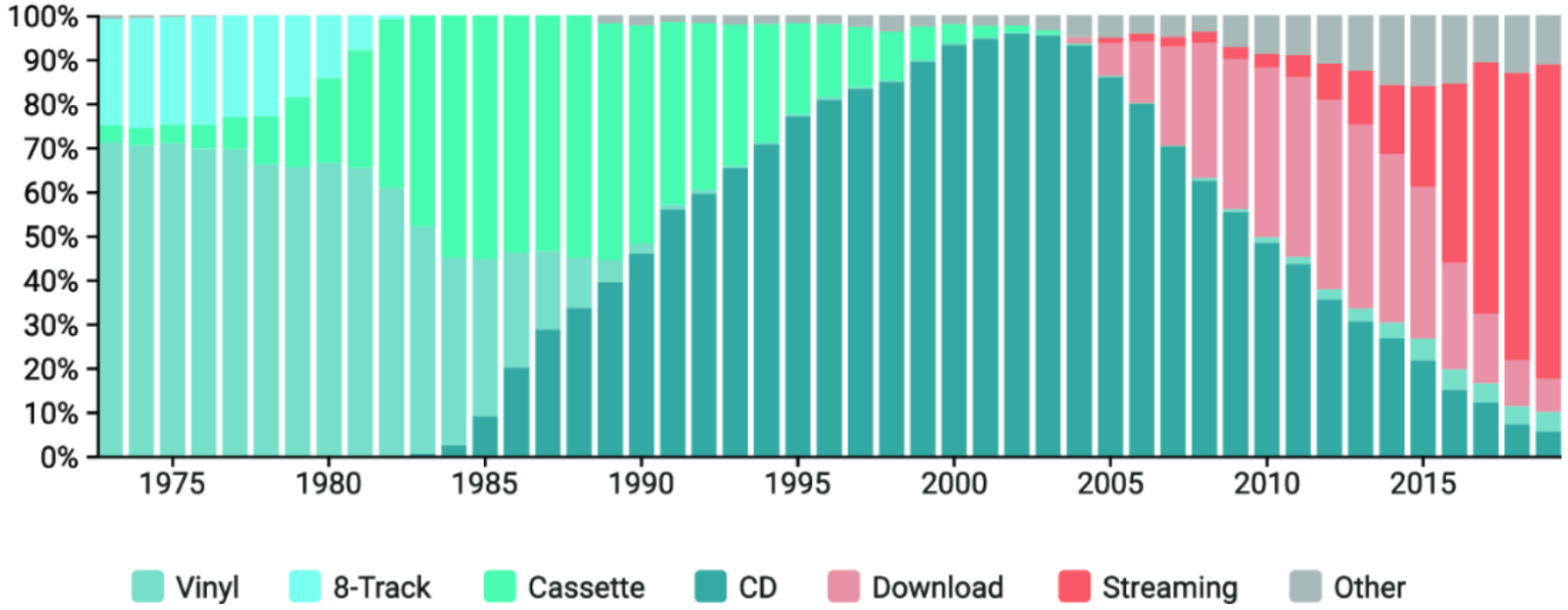


Figure 5.15 First version of the stacked bar chart



Dufour & Meeks (2024)

Stacked bar chart

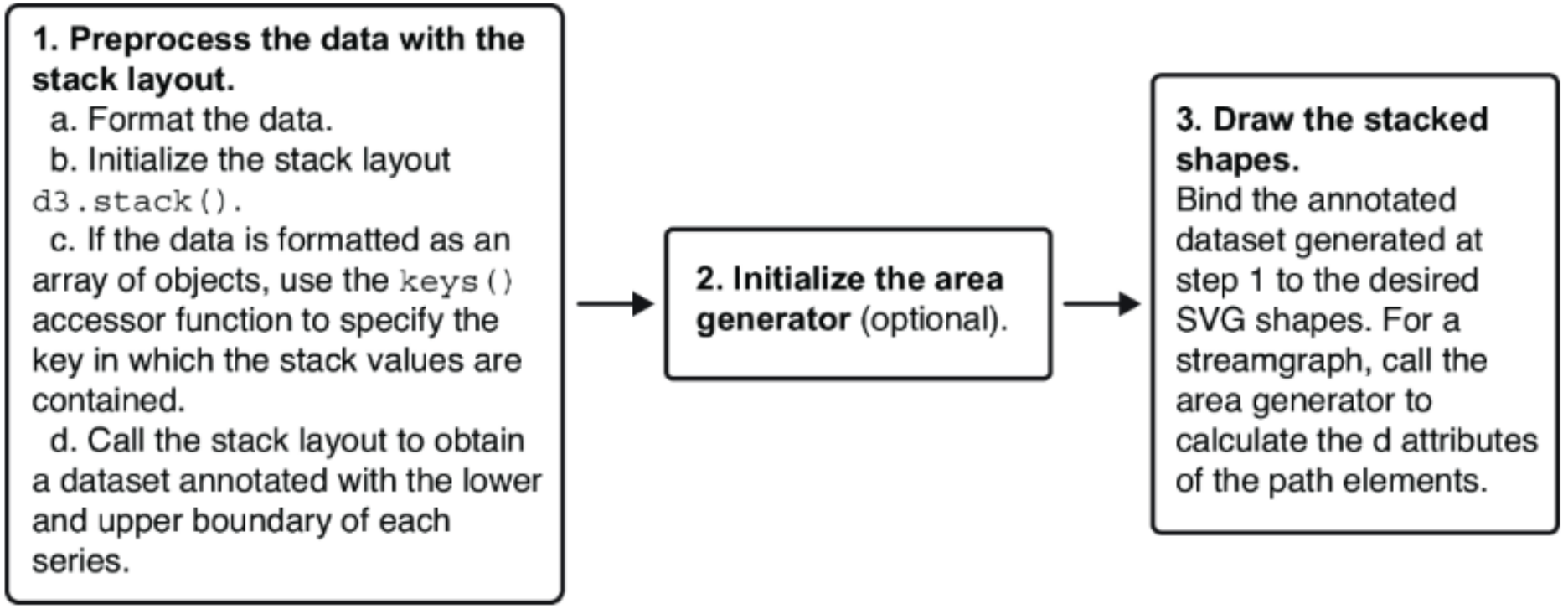


Figure 5.12 The steps to creating a stacked chart with D3

Dufour & Meeks (2024)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Add svg container and group for chart

Start in new js file (stacked-bars.js)

```
const drawStackedBars = (data) => {
        const svg = d3.select("#bars")
          .append("svg")
           .attr("viewBox", [0, 0, width, height]);

        const innerChart = svg
          .append("g")
           .attr("transform", `translate(${margin.left}, ${margin.top})`);


};
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Stacked bar chart

Declare a stackGenerator

To get categories of data that need to be stacked:

- Accessor function
- keys()

Makes an array from category data (i.e., meat types)

shared-constants.js

```
const formatsInfo = [
  {id: "beef", label: "Beef", color: "#76B6C2"},
  {id: "pig", label: "Pork", color: "#4CDDF7"},
  {id: "poultry", label: "Poultry", color: "#ED7864"},
  {id: "sheep", label: "Sheep", color: "#2F8999"},
];
```

stacked-bar.js

```
const stackGenerator = d3.stack()
  .keys(formatsInfo.map(f => f.id));
```

```
["beef", "pig", "poultry", "sheep"]
```

. . . . . .

. . . . . .

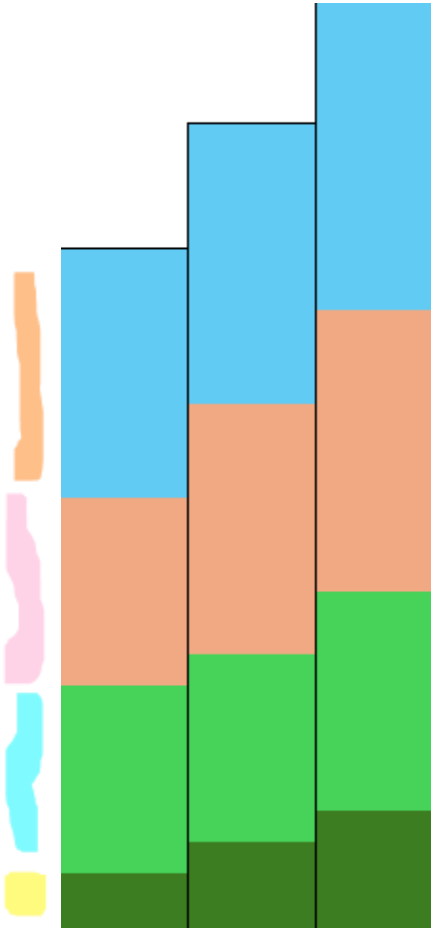
```
const annotatedData = stackGenerator(data);
```

Dufour & Meeks (2024). GitHub Copilot

SWINBURNE UNIVERSITY OF TECHNOLOGY

Stacked bar chart – annotated data set

```
{
  key: "beef",
  values: [
    { year: 1991, start: 0, end: 10 }, // Beef starts at 0, ends at 10
    { year: 1992, start: 0, end: 15 }, // Beef starts at 0, ends at 15
    { year: 1993, start: 0, end: 20 }  // Beef starts at 0, ends at 20
  ]
},
// Layer for "pig"
{
  key: "pig",
  values: [
    { year: 1991, start: 10, end: 30 }, // Pig starts at 10, ends at 30
    { year: 1992, start: 15, end: 40 }, // Pig starts at 15, ends at 40
    { year: 1993, start: 20, end: 50 }  // Pig starts at 20, ends at 50
  ]
},
// Layer for "poultry"
{
  key: "poultry",
  values: [
    { year: 1991, start: 30, end: 60 }, // Poultry starts at 30, ends at 60
    { year: 1992, start: 40, end: 75 }, // Poultry starts at 40, ends at 75
    { year: 1993, start: 50, end: 90 }  // Poultry starts at 50, ends at 90
  ]
},
// Layer for "sheep"
{
  key: "sheep",
  values: [
    { year: 1991, start: 60, end: 100 }, // Sheep starts at 60, ends at 100
    { year: 1992, start: 75, end: 120 }, // Sheep starts at 75, ends at 120
    { year: 1993, start: 90, end: 140 }  // Sheep starts at 90, ends at 140
  ]
}
];
```



1991

```
d3.stack()
  .keys([
    "vinyl",
    "eight_track",
    "cassette",
    "cd",
    "download",
    "streaming",
    "other"
  ]);
```

```
annotatedData = [
  {
    key: "vinyl",
    [
      {
        0: 0,          lower boundary (y0)
        1: 8268.5,     upper boundary (y1)
        data: { year: 1973, vinyl: 8268.5,
                eight_track: 2815.6, ... }
      },
      ...
      {
        0: 0,          lower boundary (y0)
        1: 504.4,      upper boundary (y1)
        data: { year: 2019, vinyl: 504.4,
                eight_track: 0, ... }
      }
    ]
  },
  {
    key: "eight_track",
    [
      {
        0: 8268.5,     lower boundary (y0)
        1: 11084.2,    upper boundary (y1)
        data: { year: 1973, vinyl: 8268.5,
                eight_track: 2815.6, ... }
      },
      ...
    ]
  },
  {
    key: "cassette",
    [ ... ]
  },
  {
    key: "cd",
    [ ... ]
  },
  ...
];
```

Figure 5.13 Annotated dataset returned by the stack layout generator

Dufour & Meeks (2024) , GitHub Copilot

Stack chart

Stack layout

- data columns (upper and lower boundary)
- Upper boundary: d[1]
- Lower boundary: d[2]

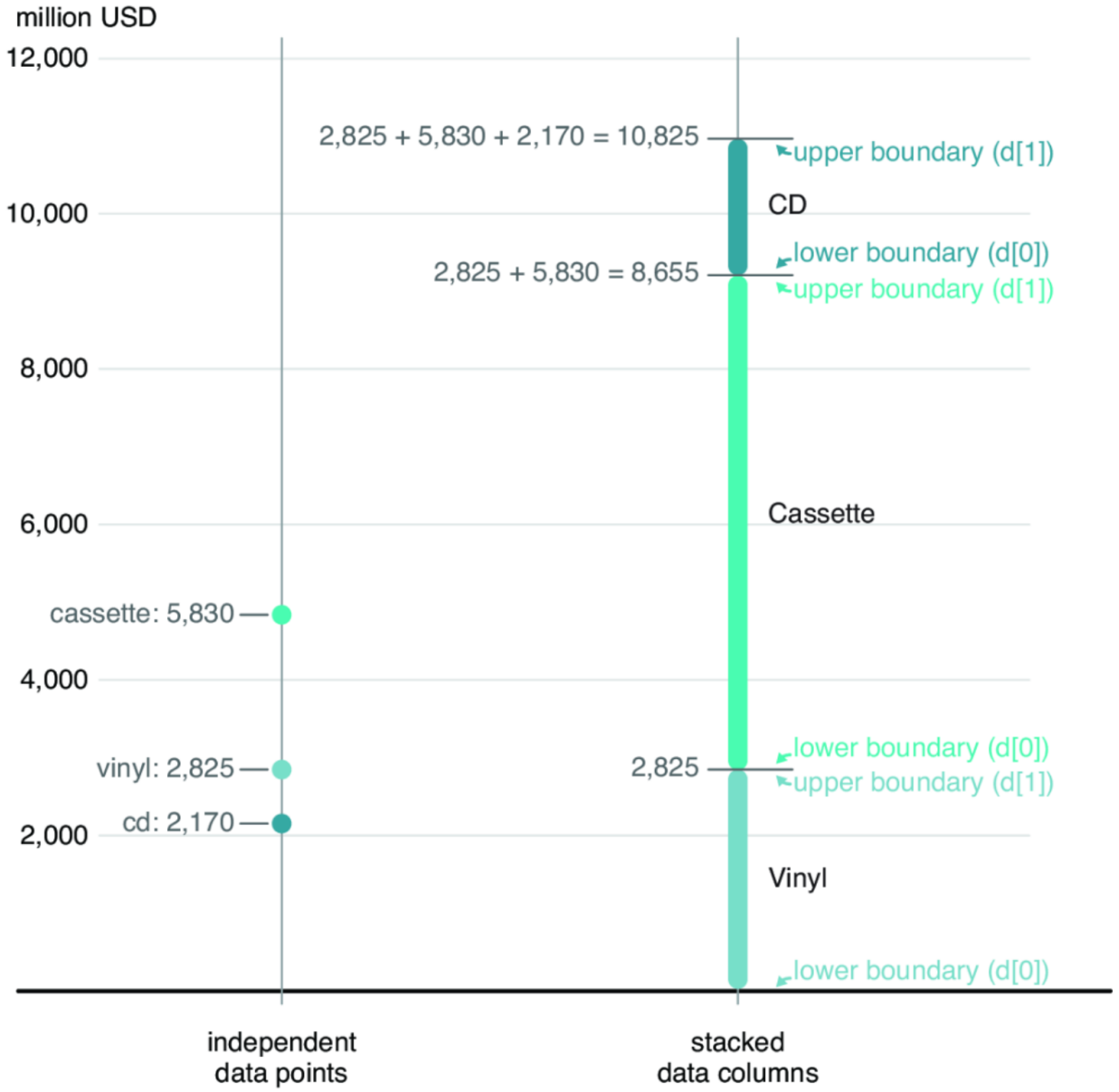


Figure 5.14 The stack layout generator transforms data points into stacked data columns and returns an annotated dataset containing each data column's lower and upper boundaries. Here we see an example for the year 1986.

Dufour & Meeks (2024)

Stack chart – y scale

Need access to annotated data set

Call within **drawStacked-Bars()**

Domain max: use max upper boundary from annotated data set

Use **nice()** to round value

```
    key: "poultry",
    values: [
      { year: 1991, start: 30, end: 60 }, // Poultry starts at 30, ends at 60
      { year: 1992, start: 40, end: 75 }, // Poultry starts at 40, ends at 75
      { year: 1993, start: 50, end: 90 }  // Poultry starts at 50, ends at 90
    ]
  },
  // Layer for "sheep"
  {
    key: "sheep",
    values: [
      { year: 1991, start: 60, end: 100 }, // Sheep starts at 60, ends at 100
      { year: 1992, start: 75, end: 120 }, // Sheep starts at 75, ends at 120
      { year: 1993, start: 90, end: 140 }  // Sheep starts at 90, ends at 140
    ]
  }
];
```

Illustrative data set

Last item (e.g., sheep)

End point

```
const maxUpperBoundary = d3.max(annotatedData[annotatedData.length - 1], d => d[1]);

const yScale = d3.scaleLinear()
  .domain([0, maxUpperBoundary])
  .range([innerHeight, 0])
  .nice();
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Stack chart

For each series of annotated dataset

Create rectangles for each data point in the series, assigning a unique class name generated by **bar-{series.key}** to ensure each rectangle is associated with its respective data series and does not get replaced during updates.

Use scales to position and fill rectangles

```
annotatedData.forEach(series => {

  innerChart
    .selectAll(`.bar-${series.key}`)
    .data(series)
    .join("rect")
    .attr("class", d => `bar-${series.key}`)

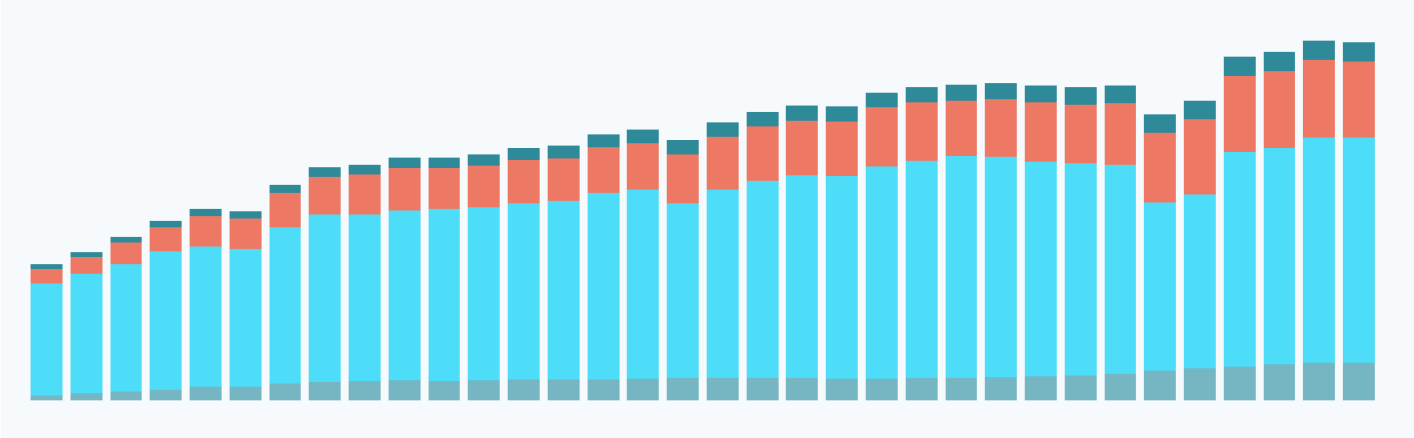
    .attr("x", d => xScale(d.data.year))
    .attr("y", d => yScale(d[1]))
    .attr("width", xScale.bandwidth())
    .attr("height", d => yScale(d[0]) - yScale(d[1]))
    .attr("fill", colorScale(series.key));
```

Dufour & Meeks (2024). GitHub Copilot

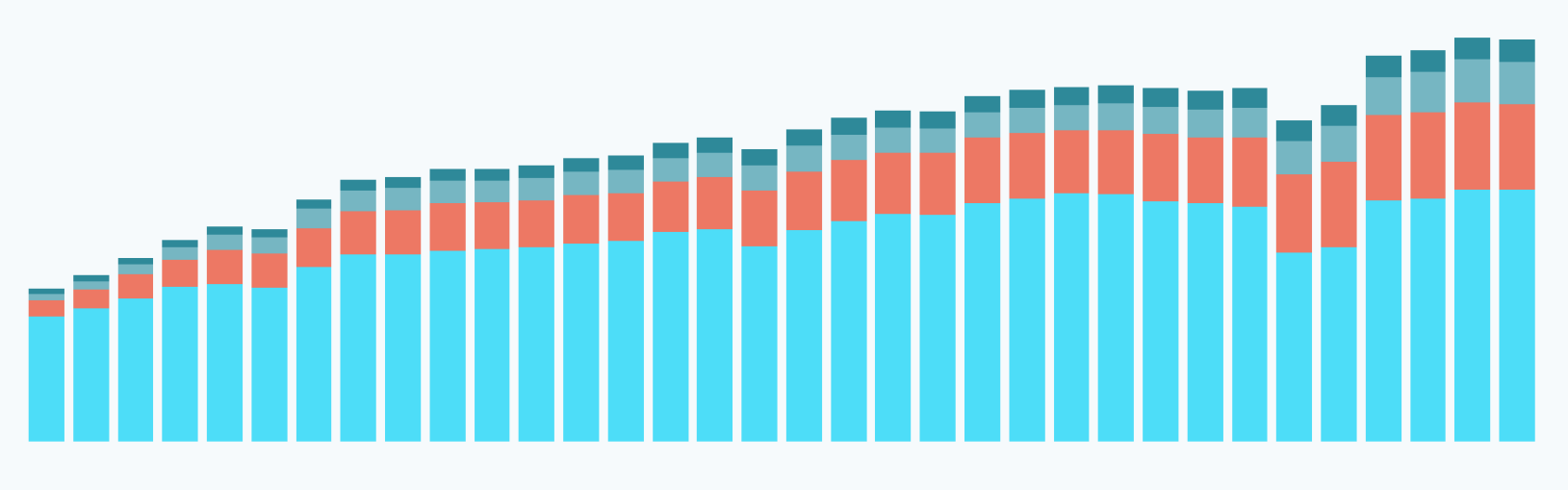
SWINBURNE UNIVERSITY OF TECHNOLOGY

Stacked bar chart

Not easy to read like this, reorder so largest values are on top…



```
const stackGenerator = d3.stack()
  .keys(formatsInfo.map(f => f.id))
  .order(d3.stackOrderDescending);
```



Dufour & Meeks (2024)

SWINBURNE UNIVERSITY OF TECHNOLOGY

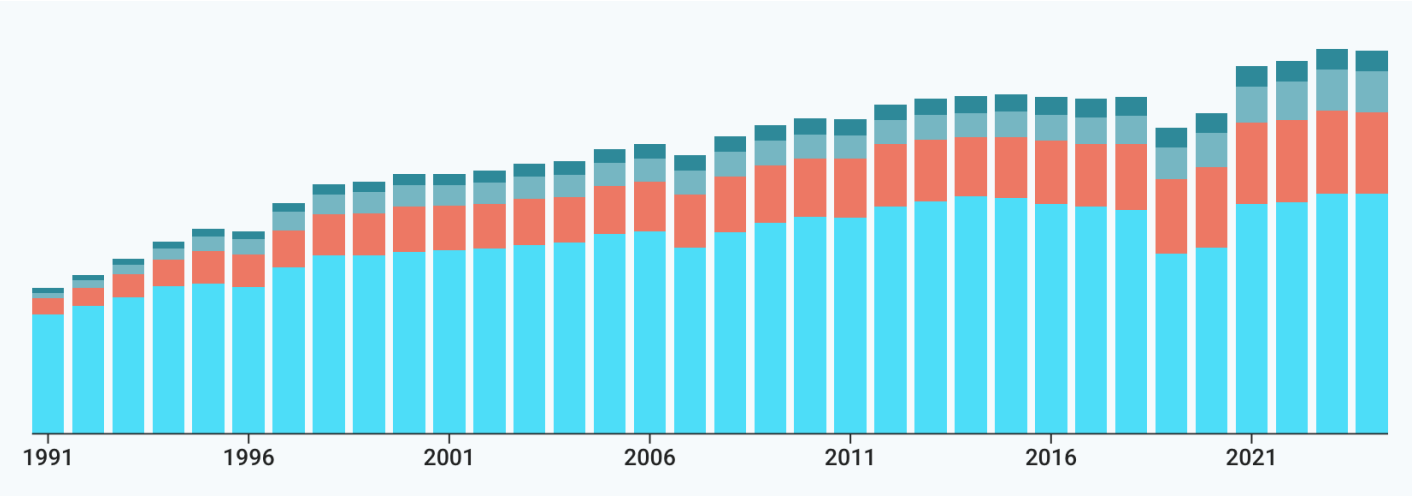
Stacked bar chart

Declare bottomAxis using xScale

Chain tickValues

Hide first tick on bottom axis

Append to the group

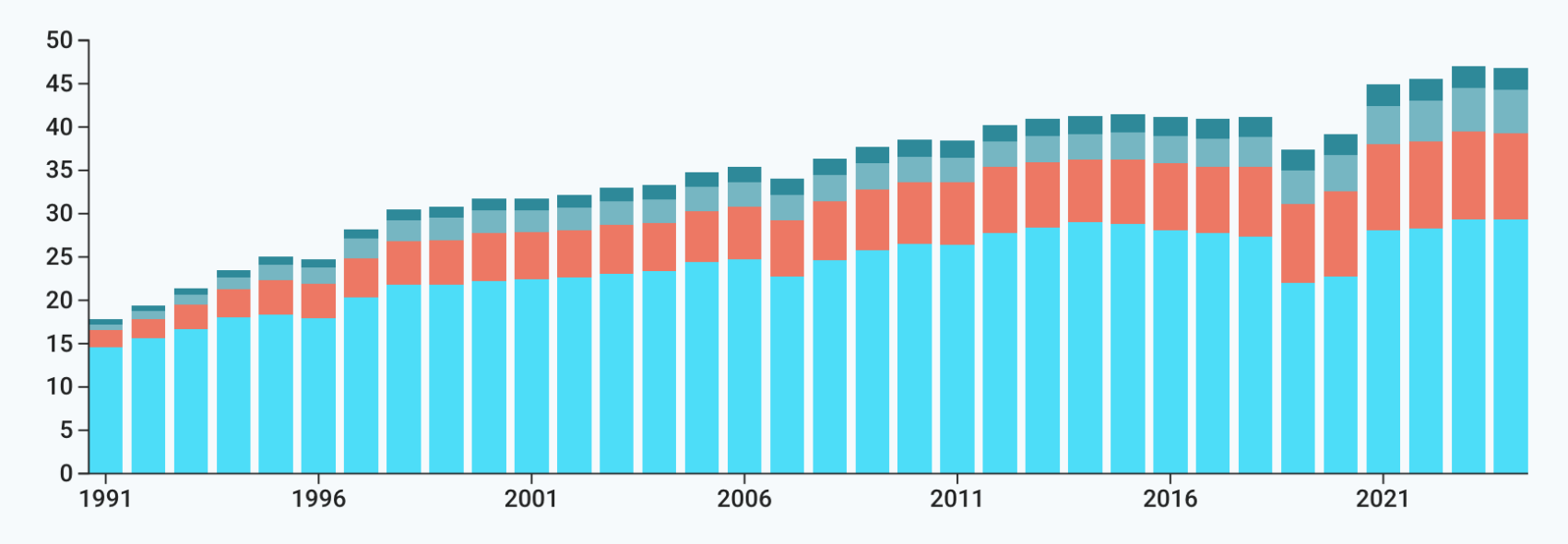



```
const bottomAxis = d3.axisBottom(xScale)
  .tickValues(d3.range(1991, 2024,5))
  .tickSizeOuter(0);

innerChart
  .append("g")
    .attr("transform", `translate(0, ${innerHeight})`)
    .call(bottomAxis);
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Stacked bar chart



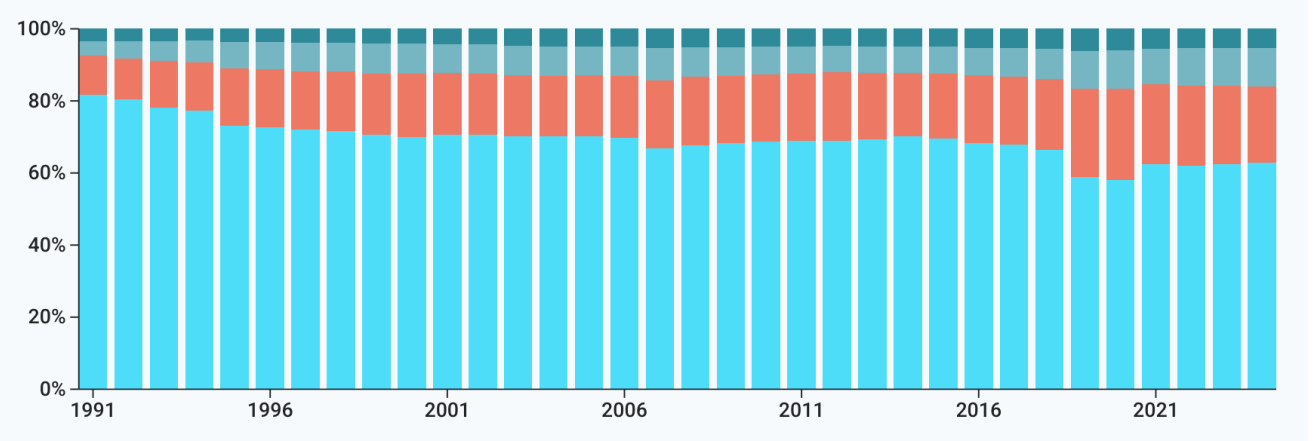

```
const leftAxis = d3.axisLeft(yScale);

innerChart
  .append("g")
  .call(leftAxis);
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Proportional stacked bar chart

```
const stackGenerator = d3.stack()
 .keys(formatsInfo.map(f => f.id))
 .order(d3.stackOrderDescending)
 .offset(d3.stackOffsetExpand);
```




```
const yScale = d3.scaleLinear()
// .domain([0, maxUpperBoundary])
  .domain([0, 1])
  .range([innerHeight, 0])
  .nice();
```

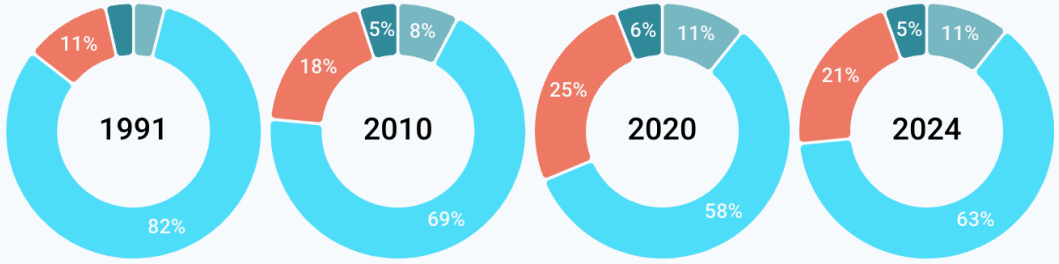
```
const leftAxis = d3.axisLeft(yScale)
  .tickFormat(d3.format(".0%"))
  .ticks(5)
  .tickSizeOuter(0);
```

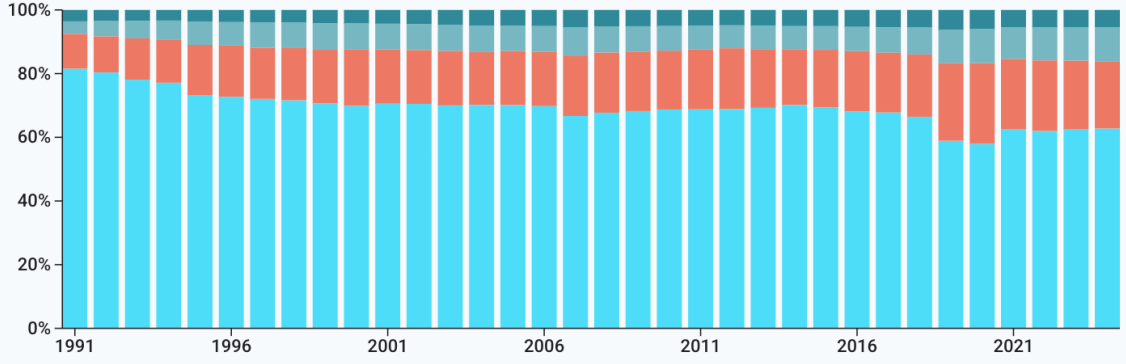
GitHub Copilot

SWINBURNE UNIVERSITY OF TECHNOLOGY

Visualizing 30 Years of Meat Consumption in China

This project visualizes the meat produced for human consumption in the China between 1991 and 2024. Data sourced from OECD for different types of meat show that pork is the perenial favorite with a bit of a dip over COVID. Next and slowly rising in popularity, particularly over COVID, is chicken, followed by beef and sheep meat.





SWINBURNE UNIVERSITY OF TECHNOLOGY